

Preparazione all'Esame Orale di Sistemi Operativi

Risposte Complete basate sul Materiale Didattico

Anno Accademico 2025/2026

Indice

1	Organizzazione e struttura di un sistema operativo, principali funzionalità	3
1.1	Principali funzionalità del SO	3
2	Evoluzione dei sistemi operativi	4
3	Passi del parsing della shell	4
4	Shell, principali comandi	5
4.1	Classificazione dei comandi	5
4.2	Principali comandi	5
5	Definizione di processo, elementi principali, stati di un processo e relative system call	6
5.1	Elementi principali di un processo	6
5.2	Stati di un processo	6
5.3	Stati di un processo aggiuntivi	6
5.4	Relative System Call (Unix)	7
6	Context switch, processi pesanti e processi leggeri	7
7	Creazione processo Unix vs creazione thread Java	7
8	Processi interagenti: sincronizzazione e comunicazione con relative system call	8
8.1	Scambio di Messaggi (Message Passing)	8
8.2	Memoria Condivisa (Shared Memory)	8
8.3	Meccanismi di Sostegno e Sincronizzazione: Semafori	9
9	Gestione segnali da parte di un processo, relative system call, segnali principali	10
9.1	Gestione dei segnali	10
9.2	Relative System Call	10
9.3	Segnali Principali	10
10	Immagine di un processo UNIX, cosa succede dopo fork/exec	11
10.1	Cosa succede dopo fork()	11
10.2	Cosa succede dopo exec()	11
11	Tipologie di scheduling	13

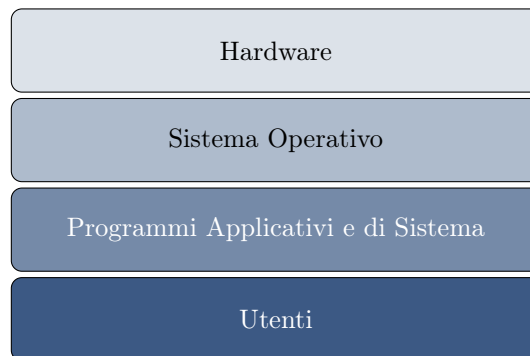
12 Criteri e algoritmi di scheduling a breve termine	13
12.1 Principali algoritmi	13
13 I 12 bit di protezione e come modificarli	14
13.1 Rappresentazione Ottale	14
13.2 Come modificarli	14
14 Metodi di allocazione di file in memoria secondaria e possibili metodi di accesso	15
14.1 Metodi di Allocazione	15
14.2 Metodi di Accesso	15
15 Unix File System	16
15.1 La Directory in Unix	16
16 Strutture dati del Kernel per accedere ai file	17
17 Ridirezione verso file (shell e system call), ridirezione verso pipe / piping (shell e system call)	18
17.1 Ridirezione verso file	18
17.2 Ridirezione verso pipe (Piping)	18
18 Metodi di allocazione dell'immagine dei processi in memoria principale	19
19 Memoria virtuale	19
20 Paginazione su richiesta	20
21 Sostituzione di pagine e identificazione di una vittima	21
21.1 Algoritmi di Sostituzione	21
22 Swapper vs pager	21
23 Working set	22
24 Organizzazione Logica e Funzioni per la Gestione dei Dispositivi	23
24.1 Componenti Hardware per l'I/O	23
24.2 Architettura del Software di I/O	23
24.3 Procedura d'Interazione Standard (Flusso I/O)	23
24.4 Funzioni del Sottosistema Indipendente dal Dispositivo	23
24.5 Tipologie di Gestione dell'I/O	24
25 Politiche di scheduling per l'accesso all'Hard Disk	24
26 RAID: Redundant Array of Independent Disks	25
27 Protezione	27

1 Organizzazione e struttura di un sistema operativo, principali funzionalità

Un sistema operativo (SO) è un programma che agisce come intermediario tra l'utente di un calcolatore e l'hardware dell'elaboratore. Ha due scopi fondamentali: fornire un ambiente confortevole per l'esecuzione dei programmi utente e allocare le risorse del computer in modo efficiente.

L'organizzazione di un sistema di calcolo prevede una struttura gerarchica (a strati o *a cipolla*), le cui componenti principali sono:

- **Hardware:** fornisce le risorse di calcolo base (CPU, memoria, dispositivi di I/O).
- **Sistema Operativo:** controlla e coordina l'uso dell'hardware tra i vari programmi applicativi per conto dei diversi utenti.
- **Programmi applicativi** (e di sistema): definiscono i modi in cui le risorse del sistema sono utilizzate per risolvere i problemi computazionali degli utenti (compilatori, database, browser, shell).
- **Utenti:** persone, macchine o altri computer.



1.1 Principali funzionalità del SO

1. **Gestione dei processi:** creazione, cancellazione, sospensione, riattivazione e sincronizzazione dei processi.
2. **Gestione della memoria principale:** tracciamento delle parti di memoria utilizzate, allocazione e deallocazione dello spazio per i programmi in esecuzione.
3. **Gestione dei file:** creazione e cancellazione di file/directory, supporto a primitive per la loro manipolazione e mappatura sulla memoria secondaria.
4. **Gestione del sistema di I/O:** mascheramento delle peculiarità hardware dei dispositivi tramite un sottosistema di buffering, caching e driver specifici.
5. **Gestione della memoria secondaria:** gestione dello spazio libero, allocazione della memoria di massa e scheduling del disco.
6. **Protezione e sicurezza:** controllo dell'accesso alle risorse da parte di processi o utenti autorizzati.

2 Evoluzione dei sistemi operativi

I sistemi operativi si sono evoluti nel tempo per massimizzare l'utilizzo dell'hardware (inizialmente molto costoso) e migliorare l'esperienza utente:

- **Sistemi a lotti** (*Batch Systems*): i programmi venivano raccolti su schede perforate o nastri da un operatore e caricati in sequenza (un lotto alla volta) per evitare tempi morti. La CPU rimaneva spesso inattiva durante le operazioni di I/O.
- **Multiprogrammazione** (*Multiprogramming*): introdotta per aumentare l'efficienza della CPU. Più lavori (*jobs*) vengono mantenuti contemporaneamente in memoria principale. Quando il lavoro in esecuzione deve attendere un'operazione di I/O, il SO commuta la CPU su un altro lavoro, garantendo che la CPU non rimanga mai inattiva.
- **Sistemi a divisione di tempo** (*Time-Sharing / Multitasking*): un'estensione logica della multiprogrammazione in cui la CPU esegue più lavori commutando tra di essi con una frequenza così elevata da consentire l'interazione diretta con ciascun utente durante l'esecuzione del programma. Il tempo di risposta deve essere breve (tipicamente inferiore a 1 secondo).

3 Passi del parsing della shell

La shell è un interprete di comandi che funge da interfaccia tra utente e sistema operativo. Quando l'utente digita una riga di comando e preme Invio, la shell esegue una sequenza strutturata di passi di parsing prima dell'esecuzione vera e propria:

1. **Lettura e Tokenizzazione**: legge la stringa di input e la scompone in parole (*token*) usando gli spazi vuoti, i tabulatori e i caratteri speciali (come `;`, `|`, `<`, `>`, `&`) come delimitatori.
2. **Espansione delle variabili ed espansione della home**: sostituisce i riferimenti alle variabili di ambiente (es. `$VAR` o `${VAR}`) con il loro valore corrente e trasforma il carattere `~` nel percorso assoluto della home directory dell'utente.
3. **Sostituzione dei comandi**: se sono presenti costrutti del tipo `'comando'` o `$(comando)`, la shell esegue preventivamente quel comando e sostituisce l'intero costrutto con il testo prodotto dal suo standard output.
4. **Filename Generation** (Wildcards / Globbing): interpreta i caratteri speciali come `*`, `?` e `[...]` per espanderli nell'elenco alfabetico dei nomi dei file corrispondenti presenti nel file system (es. `*.txt` o `doc?.txt`).
5. **Rilevamento e rimozione delle ridirezioni**: cerca i caratteri `<`, `>`, `>>`, `2>` per configurare i file di input/output. Questi token e i relativi argomenti vengono elaborati e rimossi dalla lista dei parametri del comando finale.
6. **Esecuzione del comando**: identifica il primo token rimasto come il nome del comando (eseguibile sul disco o comando interno) ed effettua la chiamata di sistema (`fork` ed `exec`) passando i restanti token come array di argomenti (`argv`).

4 Shell, principali comandi

La shell consente di navigare nel file system, gestire file e processi.

4.1 Classificazione dei comandi

1. **Comandi Built-in:** sono comandi integrati, eseguiti direttamente all'interno dello spazio della shell corrente (es. `cd`). Hanno la capacita' di modificarne direttamente l'ambiente di esecuzione.
2. **Comandi Esterni (Eseguibili):** Sono veri e propri programmi binari residenti sul disco, dentro cartelle di sistema (es. `ls`, `cat`, `cp`). Quando digiti un comando, la shell cerca l'eseguibile consultando la variabile d'ambiente `$PATH`, che contiene l'elenco di queste directory ordinate per priorita' (es. `bin/`, `sbin/`). Per eseguirli, la shell deve istanziare un nuovo processo separato combinando le chiamate di sistema `fork()` ed `execvp()`.

4.2 Principali comandi

- `ls`: elenca il contenuto di una directory (`ls -l` mostra i dettagli e i permessi).
- `cd`: cambia la directory corrente del processo della shell.
- `pwd`: stampa il percorso assoluto della directory di lavoro corrente.
- `cp` / `mv` / `rm`: rispettivamente copiano, spostano/rinominano e rimuovono (cancellano) file o directory.
- `mkdir` / `rmdir`: creano e rimuovono directory vuote.
- `cat`: concatena e mostra il contenuto di uno o più file a video.
- `chmod`: modifica i permessi di accesso (i 12 bit di protezione) di un file o directory.
- `ps`: mostra l'elenco dei processi attivi nel sistema.
- `kill`: invia un segnale (di default `SIGTERM`, o `SIGKILL` tramite il flag `-9`) a un processo specificando il suo PID.

5 Definizione di processo, elementi principali, stati di un processo e relative system call

Un processo è un programma in esecuzione. Rappresenta l'unità di lavoro fondamentale in un moderno sistema operativo multiprogrammato.

5.1 Elementi principali di un processo

Un processo è caratterizzato da un'immagine in memoria che include il contatore di programma (PC), i registri della CPU, lo stack (per dati temporanei come parametri di funzioni e variabili locali) e la sezione dati/heap (variabili globali e memoria allocata dinamicamente). Tutte queste informazioni sono memorizzate dal kernel all'interno di una struttura dati chiamata **Process Control Block (PCB)** o *process descriptor*.

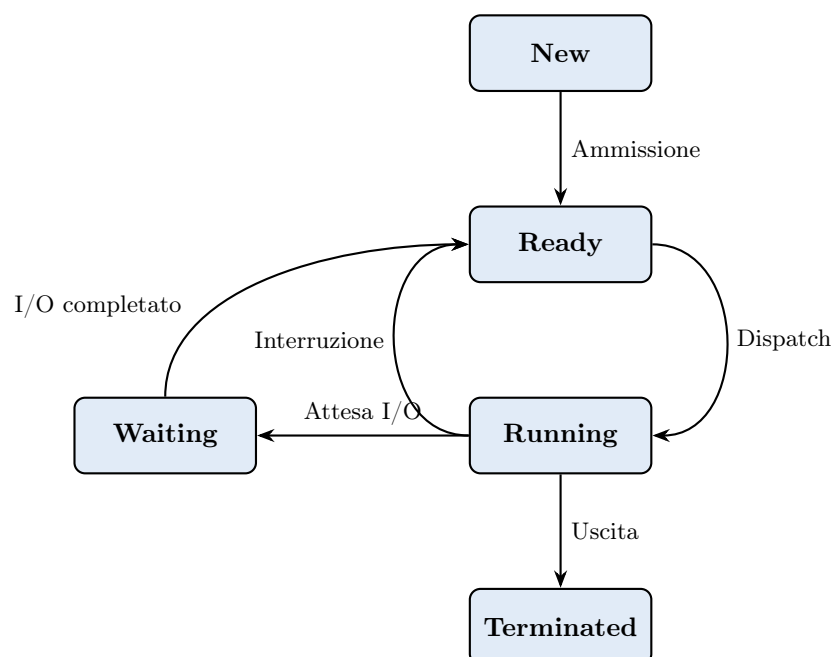
5.2 Stati di un processo

Durante la sua vita, un processo cambia stato in base agli eventi del sistema:

1. **New** (Nuovo): il processo è in fase di creazione.
2. **Ready** (Pronto): il processo è in memoria ed è in attesa di essere assegnato alla CPU dallo scheduler.
3. **Running** (In esecuzione): le istruzioni del processo vengono eseguite dalla CPU.
4. **Waiting/Blocked** (In attesa): il processo è in attesa del verificarsi di un evento esterno (es. completamento di un I/O, ricezione di un segnale).
5. **Terminated** (Terminato): il processo ha completato la sua esecuzione.

5.3 Stati di un processo aggiuntivi

- **Zombie**: processo è terminato, ma è in attesa che il padre ne rilevi lo stato di terminazione.
- **Swapped**: processo (o parte di esso) è temporaneamente trasferito in memoria secondaria



5.4 Relative System Call (Unix)

- **fork()**: crea un nuovo processo figlio duplicando il processo padre.
- **exit()**: termina volontariamente l'esecuzione del processo corrente.
- **wait()** / **waitpid()**: sospende il processo chiamante (mettendolo in stato Waiting) finché uno dei suoi processi figli non termina.

6 Context switch, processi pesanti e processi leggeri

- **Context Switch** (Cambio di contesto): è il meccanismo con cui il kernel commuta il controllo della CPU da un processo a un altro. Quando avviene un'interruzione (es. timer dello scheduling), il kernel deve salvare lo stato (contesto) del processo attualmente in esecuzione nel suo PCB e caricare lo stato precedentemente salvato del nuovo processo selezionato dal suo rispettivo PCB. Il tempo di context switch rappresenta un puro overhead poiché la CPU non esegue lavoro utile per gli utenti durante la commutazione.
- **Processi Pesanti** (*Heavyweight Processes*): corrispondono al concetto classico di processo Unix tradizionale. Possiedono un proprio spazio di indirizzamento isolato, un proprio set di descrittori di file e risorse separate. Il context switch tra processi pesanti è costoso poiché richiede di cambiare la mappa della memoria virtuale (svuotamento del TLB, ecc.).
- **Processi Leggeri** (*Lightweight Processes / Thread*): rappresentano un'unità base di utilizzo della CPU all'interno di un processo. Più thread appartenenti allo stesso processo condividono la stessa sezione di codice, la sezione dati, l'heap e le risorse del sistema operativo (come i file aperti), ma mantengono ciascuno un proprio contatore di programma (PC), i registri e un proprio stack privato. Il context switch tra thread dello stesso processo è molto più veloce rispetto a quello tra processi pesanti perché non richiede la commutazione dello spazio di indirizzamento della memoria.

7 Creazione processo Unix vs creazione thread Java

- **Creazione Processo Unix (fork)**: in Unix, un nuovo processo si crea tramite la chiamata di sistema **fork()**. Questa operazione duplica il processo chiamante creando una copia (logicamente) esatta del padre (stesso codice, dati, stack e descrittori di file, ma in uno spazio di memoria distinto e isolato). L'unica distinzione immediata è il valore di ritorno della **fork()**: restituisce 0 al processo figlio e il PID del figlio al processo padre. Il figlio eredita i dati ma le modifiche successive del figlio non influenzano il padre.
- **Creazione Thread Java**: in Java, la creazione di un'unità di esecuzione concorrente avviene interamente nello spazio utente gestito dalla Java Virtual Machine (JVM). Si realizza istanziando un oggetto che implementa l'interfaccia **Runnable** o estende la classe **Thread**, implementando il metodo **run()** e invocando successivamente il metodo **start()**. A differenza di **fork()**, il nuovo thread Java non copia l'intera memoria del chiamante, ma condivide direttamente lo stesso heap e gli stessi oggetti in memoria del processo della JVM che lo ha generato, disponendo solo di uno stack privato per le chiamate ai metodi.

8 Processi interagenti: sincronizzazione e comunicazione con relative system call

I processi che cooperano per completare un compito o condividere risorse devono interagire tra loro. Tale interazione richiede due aspetti fondamentali:

- **Comunicazione:** lo scambio effettivo di dati o informazioni tra gli spazi di memoria dei processi.
- **Sincronizzazione:** il coordinamento dei tempi di esecuzione per evitare che l'accesso concorrente a risorse condivise generi incoerenze (condizioni di corsa o *race condition*).

Per la comunicazione si adottano due modelli principali: lo *Scambio di Messaggi* e la *Memoria Condivisa*.

8.1 Scambio di Messaggi (Message Passing)

Nel modello a scambio di messaggi, i processi comunicano scambiandosi esplicitamente pacchetti di dati tramite primitive del kernel, senza condividere alcuno spazio di memoria virtuale.

La sincronizzazione in questo modello è **implicita**: se un processo tenta di leggere da un canale vuoto, il kernel lo mette automaticamente in attesa fino all'arrivo dei dati.

- **System Call per Pipe (Unix):** la chiamata di sistema `pipe(int fd[2])` crea un canale di comunicazione unidirezionale nel kernel.
- **Gestione dei file descriptor:** l'array memorizza due descrittori: `fd[0]` è aperto in lettura, mentre `fd[1]` è aperto in scrittura.
- **Operazioni di I/O:** il trasferimento dei dati avviene tramite le canoniche chiamate di sistema `write(fd[1], buf, size)` per inserire i dati nel canale e `read(fd[0], buf, size)` per estrarli.

8.2 Memoria Condivisa (Shared Memory)

Nel modello a memoria condivisa, due o più processi concordano di mappare la medesima regione di memoria fisica RAM all'interno dei rispettivi spazi di indirizzamento virtuale.

L'accesso ai dati è estremamente rapido poiché le operazioni avvengono alla velocità della memoria centrale, senza che il kernel debba fare da intermediario durante la lettura/scrittura. Tuttavia, questo modello offre solo la comunicazione e **non fornisce sincronizzazione automatica**: l'accesso simultaneo non regolamentato produce *race condition*.

Le chiamate di sistema principali (standard System V / POSIX) sono:

- `shmget()`: crea un nuovo segmento di memoria condivisa o ottiene l'identificatore di uno esistente.
- `shmat()`: (*attach*) mappa e collega il segmento di memoria condivisa allo spazio di indirizzamento virtuale del processo chiamante.
- `shmdt()`: (*detach*) scollega il segmento dal relativo spazio di indirizzamento quando l'accesso non è più necessario.

8.3 Meccanismi di Sostegno e Sincronizzazione: Semafori

Per prevenire le *race condition* nell'uso della memoria condivisa o per ordinare le esecuzioni, si usano i **semafori**. Un semaforo S è una variabile intera protetta dal kernel e accessibile esclusivamente tramite due operazioni atomiche (ossia indivisibili e ininterrompibili):

- **wait(S) (o operazione P)**: decrementa il valore del semaforo di 1 ($S = S - 1$). Se il valore diventa negativo ($S < 0$), il processo chiamante viene bloccato e inserito nella coda d'attesa del semaforo.
- **signal(S) (o operazione V)**: incrementa il valore del semaforo di 1 ($S = S + 1$). Se vi sono processi sospesi nella coda ($S \leq 0$), uno di essi viene risvegliato dal kernel e rimesso in stato di *Ready*.

I semafori si dividono principalmente in due categorie d'uso:

1. **Semaforo Binario (Mutex)**: inizializzato a 1, viene utilizzato per garantire la *mutua esclusione* nell'accesso alla Sezione Critica.
2. **Semaforo di Conteggio**: inizializzato a N , usato per gestire l'accesso a risorse disponibili in molteplici istanze o per sincronizzare l'ordine degli eventi tra processi.

Le System Call Unix (System V) per gestire i semafori includono:

- **semget()**: per creare o accedere a un insieme (set) di semafori.
- **semop()**: per eseguire in modo atomico le operazioni di **wait** e **signal** sui semafori.

9 Gestione segnali da parte di un processo, relative system call, segnali principali

Un segnale è una notifica asincrona inviata a un processo per informarlo del verificarsi di un determinato evento hardware o software.

9.1 Gestione dei segnali

Quando un segnale viene recapitato a un processo, quest'ultimo può reagire in tre modi:

1. **Azione di default:** il sistema operativo esegue l'azione predefinita associata al segnale (es. terminare il processo, ignorarlo, sospenderlo).
2. **Ignorare il segnale:** il segnale viene scartato (tranne alcuni segnali non mascherabili).
3. **Catturare il segnale** (Signal Handler): il processo esegue una specifica funzione utente (il gestore) interrompendo temporaneamente il normale flusso di esecuzione. Una volta completato il gestore, il processo riprende da dove era stato interrotto.

9.2 Relative System Call

- `kill(pid_t pid, int sig)`: invia il segnale `sig` al processo `pid`.
- `signal(int signum, sighandler_t handler)`: associa una funzione di gestione (handler) al segnale `signum`.
- `sigaction(int signum, const struct sigaction *act, struct sigaction *oldact)`: versione evoluta e robusta della precedente. Permette di configurare una maschera di segnali (`sa_mask`) per bloccare temporaneamente la ricezione di altri segnali durante l'esecuzione dell'handler stesso, impedendo perdite di dati o annidamenti incontrollati.
- `alarm(unsigned int seconds)`: programma l'invio del segnale `SIGALRM` al processo chiamante dopo lo scadere dei secondi indicati.

9.3 Segnali Principali

- **SIGINT (2)**: generato da tastiera tramite Ctrl+C. Richiede l'interruzione del processo.
- **SIGKILL (9)**: terminazione immediata e forzata. Non può essere catturato, ignorato o bloccato dal processo.
- **SIGSEGV (11)**: errore di segmentazione (accesso a memoria non valida).
- **SIGALRM (14)**: inviato dal timer attivato tramite la system call `alarm`.
- **SIGCHLD (17)**: inviato al processo padre ogni volta che un figlio termina o si interrompe.
- **SIGUSR1(10) E SIGUSR2(12)**: segnali personalizzati dall'utente gestiti dallo sviluppatore tramite un handler custom per scatenare eventi specifici nell'applicazione.
- **SIGTERM (15)**: richiesta di terminazione elegante/gentile (è il segnale inviato di default dal comando `kill`). A differenza di `SIGKILL`, questo segnale può essere catturato o ignorato, consentendo al processo di salvare lo stato e liberare le risorse prima di chiudersi.

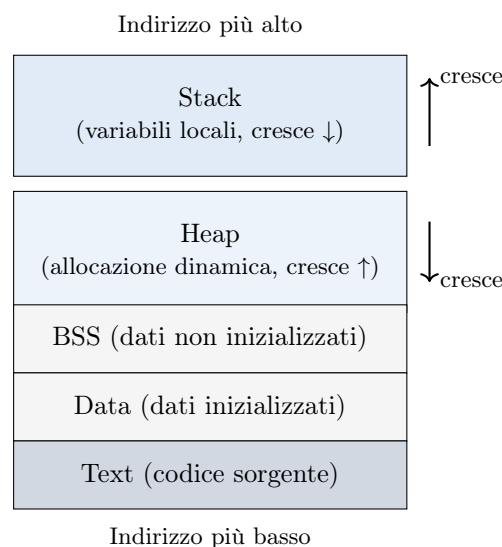
Nota

I numeri dei segnali indicati (es. `SIGINT=2`, `SIGKILL=9`, `SIGSEGV=11`, `SIGALRM=14`, `SIGCHLD=17`) sono quelli standard su Linux x86; su altre architetture o sistemi Unix i valori numerici possono differire leggermente, mentre i nomi restano standard POSIX.

10 Immagine di un processo UNIX, cosa succede dopo fork/exec

L'immagine di un processo UNIX in memoria è organizzata in sezioni distinte:

- **PCB** (Process Controll Block):
 - **Process Structure**: è l'elemento della process table associato al processo (kernel, residente), mantiene le informazioni necessarie per la gestione del processo, anche se questo è swapped in memoria secondaria.
 - **User Structer**: struttura dati contenente i dati necessari al kernel per la gestione del processo quando è residente (kernel, swappable), il suo contenuto è necessario solo in caso di esecuzione del processo (stato running); se il processo è soggetto a swapping, anche la user structure può essere trasferita in memoria secondaria.
- **Testo** (Code Segment): elemento della text table associato al codice del processo (kernel, residente).
- **Dati** (Data Segment): contiene le variabili globali del programma eseguito dal processo (user, swappable)
- **Stack e Heap** : aree dinamiche associate al programma eseguito (user, swappable)
- **Stack del kernel**: stack di sistema associato al processo per le chiamate a system call (kernel, swappable)



10.1 Cosa succede dopo fork()

La system call `fork()` crea un processo figlio che è una copia (logica) dell'immagine del padre. Il figlio riceve una copia dello stack, dell'heap, dei descrittori dei file aperti e dei dati. Nei sistemi moderni, questa copia viene gestita tramite la tecnica **Copy-on-Write (COW)**: padre e figlio condividono inizialmente le stesse pagine fisiche di memoria; la duplicazione effettiva di una pagina avviene solo se e quando uno dei due processi tenta di scrivervi.

10.2 Cosa succede dopo exec()

La famiglia di chiamate `exec()` (es. `execl`, `execvp`) sostituisce interamente l'immagine del processo corrente con un nuovo programma memorizzato sul disco. Lo stack, l'heap e il segmento dati correnti vengono distrutti e sovrascritti con quelli del nuovo file eseguibile. Il codice del vecchio programma cessa di esistere nel processo corrente. Tuttavia, il PID del processo non

cambia e, di norma, i descrittori dei file aperti rimangono validi (a meno che non sia stato impostato il flag *close-on-exec*).

11 Tipologie di scheduling

Lo scheduling è l'attività con cui il sistema operativo assegna le risorse di calcolo (CPU) ai vari processi nel tempo. Si divide in tre livelli a seconda dell'orizzonte temporale:

1. **Scheduling a lungo termine** (Job Scheduler): seleziona i processi dalla memoria di massa (mantenuti in una coda di sottomissione) e li carica in memoria principale nella coda dei processi pronti (Ready queue). Controlla il grado di multiprogrammazione del sistema. Viene eseguito raramente (intervalli di minuti).
2. **Scheduling a medio termine** (Swapper): rimuove temporaneamente i processi dalla memoria principale e li sposta nella memoria secondaria (operazione di swap-out), tipicamente per ridurre il grado di multiprogrammazione in situazioni di sovraccarico della memoria, per poi reintrodurli successivamente (swap-in).
3. **Scheduling a breve termine** (CPU Scheduler): seleziona un processo dalla coda dei pronti in memoria principale e gli assegna la CPU. Viene eseguito molto frequentemente (nell'ordine dei millisecondi) e deve quindi essere estremamente veloce.

12 Criteri e algoritmi di scheduling a breve termine

I criteri principali per valutare le prestazioni di un algoritmo di scheduling includono l'utilizzo della CPU, la produttività (*throughput*), il tempo di completamento (*turnaround time*), il tempo di attesa nella coda dei pronti e il tempo di risposta.

Gli algoritmi possono essere **Preemptive** (il SO può revocare la CPU al processo prima che abbia finito) o **Non-preemptive** (il processo mantiene la CPU finché non si blocca o termina).

12.1 Principali algoritmi

- **FCFS** (First-Come, First-Served): non-preemptive. Il primo processo che richiede la CPU la ottiene per primo. Semplice da implementare (coda FIFO) ma soffre dell'effetto convoglio (*convoy effect*), dove processi brevi rimangono bloccati dietro a un processo molto lungo, alzando il tempo medio di attesa.
- **SJF** (Shortest Job First): associa a ciascun processo la lunghezza del suo prossimo burst di CPU e assegna la CPU al processo con il burst più breve. È l'algoritmo ottimo poiché minimizza il tempo medio di attesa. Può essere non-preemptive o preemptive (in quest'ultimo caso noto come **SRTF** - Shortest Remaining Time First). Limite: è difficile conoscere a priori la durata del prossimo burst di CPU (viene stimata tramite medie esponenziali mobili). Può causare *starvation* dei processi lunghi.
- **Round Robin** (RR): preemptive, progettato per sistemi time-sharing. Definisce una piccola unità di tempo chiamata quanto di tempo (*time quantum*, tipicamente 10–100 ms). La CPU scorre la coda dei pronti in modo circolare erogando a ciascun processo al massimo un quanto di tempo. Se il processo supera il quanto, viene interrotto e reinserito in fondo alla coda dei pronti. Se il quanto è troppo grande, RR degrada a FCFS; se è troppo piccolo, l'overhead del context switch diventa dominante.
- **Scheduling a Priorità**: assegna a ciascun processo un numero intero indicante la priorità; la CPU viene concessa al processo con priorità più alta. Può causare il problema dell'*indefinite blocking* (starvation) per i processi a bassa priorità. Soluzione: l'invecchiamento (*aging*), che aumenta gradualmente la priorità dei processi che attendono a lungo nella coda dei pronti.

13 I 12 bit di protezione e come modificarli

Nei sistemi operativi Unix/Linux, ogni file è associato a un insieme di metadati che include un vettore di 12 bit di protezione memorizzato nell'i-node. Questi bit determinano le modalità di accesso al file e sono suddivisi in 4 gruppi da 3 bit ciascuno:

1. **Bit Speciali** (i primi 3 bit più significativi):
 - **SUID** (Set-User-ID): se impostato su un file eseguibile, quando un processo lo manda in esecuzione acquisisce temporaneamente i privilegi del proprietario del file anziché dell'utente che lo ha avviato (es. il comando `passwd`).
 - **SGID** (Set-Group-ID): analogo al SUID, ma il processo acquisisce i privilegi del gruppo proprietario del file.
 - **Sticky Bit**: sui file storicamente manteneva il testo in memoria; sulle directory moderne impedisce a un utente di cancellare o rinominare un file contenuto a meno che non ne sia il diretto proprietario (es. la cartella `/tmp`).
2. **Bit dell'utente proprietario** (User – u): 3 bit per Lettura (r), Scrittura (w), Esecuzione (x).
3. **Bit del gruppo** (Group – g): 3 bit per Lettura (r), Scrittura (w), Esecuzione (x).
4. **Bit degli altri utenti** (Others – o): 3 bit per Lettura (r), Scrittura (w), Esecuzione (x).

13.1 Rappresentazione Ottale

Ogni tripletta `rwX` viene convertita in una cifra ottale (da 0 a 7) dove `r=4`, `w=2`, `x=1`. Ad esempio, i permessi `rwXr-Xr-X` corrispondono al valore ottale 755.

13.2 Come modificarli

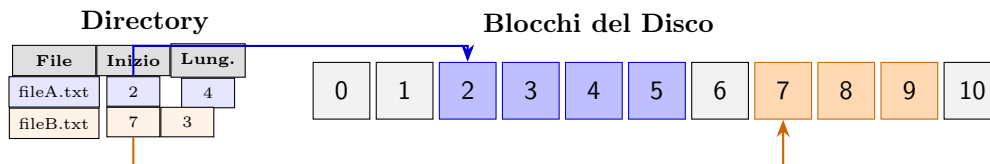
Si utilizza il comando di shell `chmod`.

- **Modalità assoluta (ottale)**: `chmod 755 nome_file` (imposta `rwXr-Xr-X`).
- **Modalità simbolica**: `chmod u+x nome_file` (aggiunge il permesso di esecuzione al proprietario), `chmod g-w nome_file` (rimuove il permesso di scrittura al gruppo).

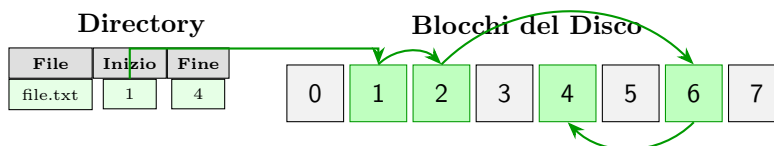
14 Metodi di allocazione di file in memoria secondaria e possibili metodi di accesso

14.1 Metodi di Allocazione

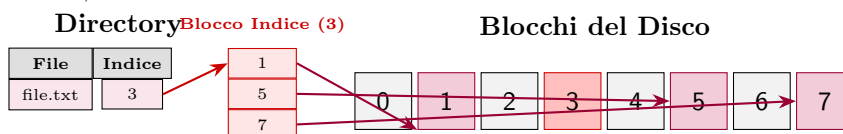
- **Allocazione Contigua:** ogni file occupa un insieme di blocchi contigui sul disco. L'accesso è estremamente rapido sia in modo sequenziale che diretto. Problemi: forte frammentazione esterna del disco e necessità di conoscere la dimensione massima del file al momento della creazione.



- **Allocazione Concatenata (Linked Allocation):** ogni file è una lista concatenata di blocchi di disco. Ogni blocco contiene i dati e un puntatore al blocco successivo. Elimina la frammentazione esterna e il file può crescere indefinitamente. Svantaggio: l'accesso è rigidamente solo sequenziale; per leggere il blocco N occorre scorrere i precedenti N-1 blocchi. Inoltre, i puntatori sottraggono spazio utile ai dati.



- **Allocazione Indicizzata:** raccoglie tutti i puntatori ai blocchi del file in un unico blocco speciale chiamato blocco indice (*index block*). Supporta l'accesso diretto efficiente senza frammentazione esterna. Tuttavia, se il file è molto piccolo c'è spreco di spazio per il blocco indice, mentre se il file è enorme occorrono schemi a indici concatenati o multilivello.



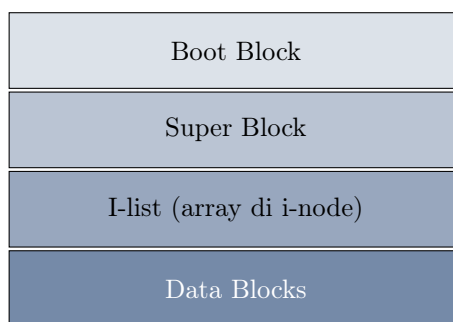
14.2 Metodi di Accesso

- **Accesso Sequenziale:** le informazioni del file vengono elaborate in ordine, un record dopo l'altro (es. editor di testo, lettori di file di log). Operazioni tipiche: `read_next()`, `write_next()`.
- **Accesso Diretto (o Casuale):** consente di accedere a qualsiasi blocco o record logico direttamente e in modo non sequenziale tramite il numero del blocco (es. i database). Operazioni tipiche: `read(n)`, `write(n)` o riposizionando il puntatore tramite `seek(n)`.

15 Unix File System

Lo Unix File System (UFS) si basa su una struttura logica ad albero e organizza il disco fisico suddividendolo in aree ben definite:

1. **Boot Block:** contiene il codice necessario ad avviare il sistema operativo (fase di bootstrap).
2. **Super Block:** contiene i metadati generali del file system (dimensione del disco, numero di blocchi liberi, puntatori alla lista dei blocchi liberi e degli i-node liberi).
3. **I-list** (Area degli i-node): un array di strutture dati chiamate i-node (index node). Esiste esattamente un i-node per ogni file o directory del sistema. L'i-node contiene tutti i metadati del file (dimensione, tipo, proprietario, bit di protezione, timestamp) eccetto il nome del file. Contiene inoltre una tabella di puntatori ai blocchi di dati del file sul disco.
4. **Data Blocks:** blocchi effettivi contenenti il contenuto dei file e delle directory.



15.1 La Directory in Unix

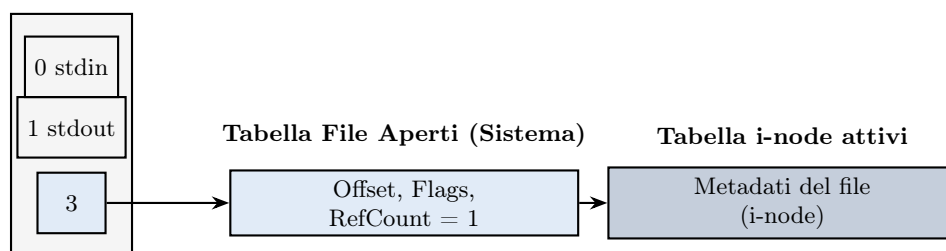
In Unix, una directory è un file speciale il cui contenuto è una tabella di coppie (Nome del file, Numero di i-node). La risoluzione di un percorso (es. `/user/ast/mbox`) avviene partendo dall'i-node della radice (`/`), leggendo i suoi blocchi dati per trovare l'i-node associato alla cartella `user`, leggendo i blocchi dati di quest'ultima per ricavare l'i-node di `ast`, e così via fino a raggiungere il file desiderato.

16 Strutture dati del Kernel per accedere ai file

Quando un processo richiede l'apertura e la manipolazione di un file, il kernel Unix coordina l'accesso attraverso tre livelli di strutture dati collegate tra loro:

1. **Tabella dei descrittori dei file per processo** (File Descriptor Table): ogni processo ha nel suo PCB un array privato di puntatori. L'indice di questo array è il File Descriptor (fd) restituito dalla system call `open()`. Ciascuna voce punta a un elemento della tabella dei file aperti del sistema.
2. **Tabella dei file aperti del sistema** (System-wide Open File Table): questa tabella tiene traccia di tutti i file attualmente aperti da qualunque processo nel sistema. Ogni voce contiene:
 - il flag di accesso (sola lettura, scrittura, ecc.);
 - l'offset corrente (la posizione corrente di lettura/scrittura all'interno del file);
 - un contatore di riferimenti (*reference count*);
 - un puntatore alla tabella degli i-node attivi nel sistema.
3. **Tabella degli i-node attivi** (In-core i-node table): contiene una copia in memoria centrale dell'i-node del file prelevato dal disco, arricchita con informazioni dinamiche (come i lock sul file).

PCB (fd table)



Nota

Grazie a questa separazione, se due processi diversi aprono indipendentemente lo stesso file, avranno due record separati nella Tabella dei File Aperti del Sistema (mantenendo quindi ciascuno il proprio offset di lettura privato), ma entrambi i record punteranno alla medesima voce nella Tabella degli i-node attivi. Di contro, dopo una `fork()`, il figlio eredita la tabella del padre, condividendo la stessa voce nella tabella di sistema e di conseguenza lo stesso offset corrente.

17 Ridirezione verso file (shell e system call), ridirezione verso pipe / piping (shell e system call)

17.1 Ridirezione verso file

- **A livello di Shell:** l'operazione `comando > file.txt` reindirizza lo standard output del comando scrivendolo nel file indicato anziché stamparlo a video.
- **A livello di System Call:** sfrutta la regola per cui il kernel alloca sempre il descrittore di file libero con il numero più basso disponibile. I passi esecutivi sono:
 1. `fork()` per creare il processo figlio.
 2. Nel figlio, si esegue `close(1)` per chiudere lo standard output.
 3. Si esegue `open("file.txt", O_WRONLY|O_CREAT, 0666)`. Poiché il descrittore 1 è appena stato liberato, il file appena aperto riceverà proprio il descrittore 1. (In alternativa si usa la system call `dup2(fd, 1)` per duplicare in modo atomico il descrittore sul canale 1.)
 4. Si esegue la `exec()`, e il nuovo programma scriverà nativamente sul canale 1, che ora punta al file.

17.2 Ridirezione verso pipe (Piping)

- **A livello di Shell:** il costrutto `comando1 | comando2` connette lo standard output di `comando1` direttamente allo standard input di `comando2`.
- **A livello di System Call:** richiede l'uso combinato di `pipe()` e `fork()`:
 1. Il processo padre crea una pipe tramite `pipe(fd)` (dove `fd[0]` è lettura e `fd[1]` è scrittura).
 2. Il padre esegue due `fork()` per generare due processi figli.
 3. Nel Figlio 1 (produttore/comando1): chiude lo standard output `close(1)`, duplica il canale di scrittura della pipe tramite `dup(fd[1])` (o `dup2`), chiude i descrittori della pipe non più necessari e lancia l'`exec(comando1)`.
 4. Nel Figlio 2 (consumatore/comando2): chiude lo standard input `close(0)`, duplica il canale di lettura della pipe tramite `dup(fd[0])`, chiude i descrittori originari e lancia l'`exec(comando2)`.
 5. Il padre chiude entrambi i lati della pipe nel proprio ambiente e attende la terminazione dei figli.

18 Metodi di allocazione dell'immagine dei processi in memoria principale

Prima dell'avvento della memoria virtuale pura, lo spazio dei processi veniva allocato in memoria fisica in modo continuo secondo tre strategie principali:

- **Partizioni Fisse:** la memoria fisica viene divisa all'avvio in partizioni di dimensione statica e prefissata. Ogni processo viene assegnato a una partizione abbastanza grande da contenerlo. Svantaggio: soffre di grave frammentazione interna (spazio sprecato all'interno della partizione se il processo è più piccolo della partizione stessa).
- **Partizioni Dinamiche:** la memoria non è divisa rigidamente, ma viene allocato dinamicamente lo spazio esatto richiesto dal processo al momento del caricamento. Col tempo, la continua creazione e terminazione di processi lascia dei buchi vuoti intervallati, creando il problema della frammentazione esterna.
 - **First-Fit:** alloca il processo nel primo blocco di memoria libero sufficientemente grande che incontra scorrendo la lista. È veloce.
 - **Best-Fit:** scorre l'intera lista e assegna il processo al blocco libero più piccolo in assoluto capace di contenerlo, riducendo al minimo lo spazio residuo del buco. Rallenta l'esecuzione e produce buchi minuscoli inutilizzabili.
 - **Worst-Fit:** assegna il processo al blocco libero più grande in assoluto, lasciando un buco residuo di grandi dimensioni potenzialmente riutilizzabile. Richiede la scansione completa.
- **Compattazione:** tecnica per risolvere la frammentazione esterna: sposta i processi attivi in memoria per raggrupparli tutti da un lato, unificando tutti i frammenti liberi in un unico grande blocco continuo. È un'operazione estremamente costosa in termini di tempo di CPU.

19 Memoria virtuale

La memoria virtuale è una tecnica che consente l'esecuzione di processi la cui immagine non è interamente contenuta nella memoria fisica. Ciò permette di avere uno spazio di indirizzamento logico nettamente superiore alla memoria RAM reale disponibile.

- **Vantaggi:** i programmi non sono più limitati dai vincoli dell'hardware fisico, è possibile caricare un numero maggiore di processi simultaneamente (aumentando il grado di multiprogrammazione) e si riduce l'I/O necessario per caricare o scambiare i processi in memoria.
- **Principio di funzionamento:** lo spazio di indirizzamento logico del processo è suddiviso in blocchi di dimensione fissa detti *pagine logiche*, mentre la memoria RAM fisica è divisa in blocchi analoghi chiamati *frame* (o cornici di pagina). Il collegamento tra i due mondi è gestito da una componente hardware della CPU chiamata **MMU** (Memory Management Unit) tramite la **Tabella delle Pagine** (Page Table). Per ogni pagina del processo, la tabella indica se la pagina si trova in un frame fisico e l'indice del frame corrispondente.

20 Paginazione su richiesta

La paginazione su richiesta (*Demand Paging*) prevede che le pagine del processo vengano caricate in memoria RAM solo quando il processo tenta effettivamente di accedervi durante l'esecuzione.

- **Bit di Validità** (Valid-Invalid Bit): nella tabella delle pagine, ogni voce dispone di un bit aggiuntivo: è impostato a *Valid* se la pagina è attualmente presente nella RAM, ed è impostato a *Invalid* se la pagina si trova ancora sulla memoria di massa (disco/area di swap) o non appartiene allo spazio del processo.
- **Page Fault** (Fallo di pagina): se il processo tenta di accedere a una pagina il cui bit è *Invalid*, la MMU interrompe l'esecuzione e solleva un'eccezione hardware detta *Page Fault trap* verso il sistema operativo.

Gestione del Page Fault da parte del SO

1. blocca il processo e salva il suo contesto;
2. verifica in una tabella interna se l'accesso alla memoria era legittimo. Se illegittimo, termina il processo (*segmentation fault*);
3. cerca un frame di pagina libero nella memoria RAM;
4. schedula un'operazione di I/O sul disco per leggere la pagina richiesta e copiarla nel frame individuato;
5. aggiorna la Tabella delle Pagine inserendo il numero del frame e commutando il bit a *Valid*;
6. ripristina il contesto del processo facendogli ripetere esattamente l'istruzione che aveva causato il fallo di pagina.

21 Sostituzione di pagine e identificazione di una vittima

Se durante la gestione di un page fault il sistema operativo rileva che tutti i frame della memoria RAM sono occupati, deve liberarne uno applicando un algoritmo di sostituzione delle pagine. La pagina rimossa viene chiamata *vittima*.

Nota

Dirty Bit (Bit di modifica): per ottimizzare la sostituzione, ogni frame ha un bit hardware che si attiva se la pagina è stata modificata in scrittura. Se la vittima ha il dirty bit a 0, significa che è identica alla copia sul disco, quindi il SO può semplicemente sovrascriverla senza fare I/O. Se è a 1, il SO deve prima scriverla sul disco (*page-out*).

21.1 Algoritmi di Sostituzione

- **FIFO** (First-In, First-Out): sostituisce la pagina che risiede in memoria da più tempo. Semplice da implementare ma può rimuovere pagine usate frequentemente. Soffre dell'**Anomalia di Belady**: per alcuni pattern di accesso, il numero di page fault può aumentare all'aumentare dei frame fisici concessi al processo.
- **Ottimo** (OPT / MIN): sostituisce la pagina che non sarà utilizzata per il più lungo periodo di tempo nel futuro. Garantisce il minor numero possibile di page fault ed è esente dall'anomalia di Belady. È un algoritmo teorico e non realizzabile poiché richiede la conoscenza perfetta del futuro. Viene usato come benchmark di confronto.
- **LRU** (Least Recently Used): approssima l'ottimo guardando al passato recente: sostituisce la pagina che non viene toccata da più tempo. Richiede supporto hardware costoso (contatori o stack aggiornati a ogni singolo accesso a memoria). Non soffre dell'anomalia di Belady.
- **Approssimazioni LRU** (Second Chance / Algoritmo dell'Orologio): sfrutta un bit di riferimento impostato a 1 dall'hardware ogni volta che la pagina viene letta/scritta. Le pagine vengono esaminate in modo circolare da una lancetta: se il bit è 1, viene azzerato e alla pagina viene data una seconda chance; se il bit è 0, la pagina viene immediatamente scelta come vittima.

22 Swapper vs pager

Entrambi si occupano del trasferimento di memoria tra RAM e memoria secondaria, ma operano a granularità differenti:

- **Swapper** (Schedulatore a medio termine): gestisce i processi nella loro interezza. Quando esegue lo swap-out, sposta l'intera immagine di un processo (tutte le sue pagine, stack, heap) fuori dalla RAM trasferendola nell'area di swap del disco per liberare spazio macroscopico.
- **Pager** (Lazy Pager): lavora a livello di singole pagine all'interno del meccanismo della memoria virtuale paginata. Non si cura dell'intero processo, ma si occupa esclusivamente di fare il page-in o il page-out di una pagina specifica alla volta in risposta ai falli di pagina o alle politiche di rimpiazzo.

23 Working set

Il modello del Working Set si basa sul principio di località: durante una determinata fase di esecuzione, un processo non accede alla memoria in modo uniforme ma si concentra in una specifica regione di pagine (località temporale e spaziale).

Definizione: il Working Set (WS) di un processo all'istante di tempo t è l'insieme delle pagine logiche accedute dal processo nell'intervallo di tempo immediatamente precedente $[t-\Delta, t]$, dove Δ è la finestra del working set (espressa in numero di riferimenti alla memoria).

Prevenzione del Thrashing: il *Thrashing* è il fenomeno degenerativo in cui il sistema spende più tempo a fare l'I/O di paginazione rispetto a quanto ne spenda per eseguire istruzioni utili, portando il throughput (numero di processi portati a termine nell'unità di tempo (es. processi/secondo)) della CPU quasi a zero. Questo accade quando la somma dei working set di tutti i processi attivi supera la RAM fisica totale disponibile:

$$\sum_i WS_i > RAM$$

Strategia del SO: il SO monitora il working set di ciascun processo. Se la memoria residua non è sufficiente a contenere l'intero working set di un processo, il SO preferisce sospendere completamente quel processo (eseguendo lo swap-out tramite lo swapper) piuttosto che lasciarlo in esecuzione con meno pagine del necessario, evitando l'insorgenza del thrashing.

24 Organizzazione Logica e Funzioni per la Gestione dei Dispositivi

L'architettura del software di I/O è organizzata in strati gerarchici per disaccoppiare l'hardware dalle applicazioni utente e garantire un'interfaccia di gestione uniforme.

24.1 Componenti Hardware per l'I/O

Ogni dispositivo hardware o periferica si interfaccia al sistema tramite un **controller** dedicato, dotato di due componenti fondamentali:

- **Buffer di dispositivo:** Un'area di memoria locale integrata nel controller in cui vengono temporaneamente depositati i dati binari letti o in fase di scrittura.
- **Registri Speciali:** Registri di controllo (per impartire comandi) e di stato (per rilevare lo stato operativo ed eventuali errori).

24.2 Architettura del Software di I/O

L'architettura logica dell'I/O si articola in strati, partendo dalle applicazioni ad alto livello fino ai componenti fisici:



24.3 Procedura d'Interazione Standard (Flusso I/O)

Quando un programma utente richiede un'operazione di lettura o scrittura, l'interazione segue una sequenza coordinata dal Sistema Operativo:

1. **Invio del Comando:** La CPU scrive nei registri speciali del controller del dispositivo per impartire le specifiche dell'operazione.
2. **Esecuzione Locale:** Il controller hardware prende in carico la richiesta e trasferisce autonomamente i dati tra il dispositivo fisico e il proprio buffer locale.
3. **Segnale di Interrupt:** A trasferimento completato, il controller invia un segnale di interruzione hardware (*interrupt*) asincrono alla CPU per notificare la fine delle operazioni.
4. **Copia in RAM:** I dati vengono infine trasferiti dal buffer del controller alla memoria centrale (RAM) affinché il processo richiedente possa accedervi.

24.4 Funzioni del Sottosistema Indipendente dal Dispositivo

Questo livello fornisce funzionalità di alto livello comuni a tutti i dispositivi, nascondendo le peculiarità dell'hardware sottostante:

- **Interfaccia Uniforme (Naming):** Assicura una mappatura coerente dei dispositivi. Nei sistemi Unix/Linux, ogni periferica riceve un nome simbolico rappresentato come un file speciale all'interno della directory `/dev` (es. `/dev/sda`).
- **Buffering:** Alloca aree di memoria temporanea per colmare le differenze di velocità o di unità di trasferimento (caratteri vs blocchi) tra i processi e l'hardware.
- **Spooling:** Gestisce l'accesso a dispositivi non condivisibili simultaneamente (es. stampanti) tramite una coda di lavoro gestita da un processo demone dedicato.
- **Gestione degli Errori:** Intercetta i malfunzionamenti dell'hardware decidendo se riprovare l'operazione in trasparenza o propagare l'errore al livello utente.
- **Protezione e Allocazione:** Verifica le autorizzazioni di accesso ai dispositivi e gestisce l'allocazione esclusiva o condivisa delle risorse di I/O.

24.5 Tipologie di Gestione dell'I/O

L'esecuzione delle operazioni di I/O da parte del Sistema Operativo può avvenire secondo due modalità:

I/O Sincrono: Il processo che ha richiesto l'operazione viene rimosso dalla CPU e posto in stato di sospensione (*waiting/sleeping*). L'esecuzione del processo rimane bloccata finché i dati non sono completamente pronti in RAM.

I/O Asincrono: Il Sistema Operativo avvia l'operazione ma restituisce immediatamente il controllo e la CPU al processo chiamante. Il processo può proseguire l'esecuzione in parallelo e, se necessario, lanciare altre richieste di I/O simultaneamente.

25 Politiche di scheduling per l'accesso all'Hard Disk

Negli hard disk meccanici, il tempo di accesso ai dati è dominato dal **Seek Time** (tempo per muovere la testina sul cilindro corretto) e dalla **Rotational Latency** (tempo di rotazione del piatto). Il SO adotta algoritmi di scheduling per ordinare la coda delle richieste di I/O pendenti in modo da minimizzare gli spostamenti totali della testina.

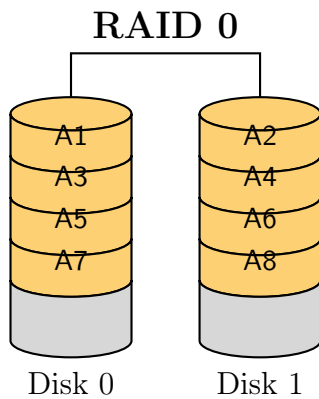
Esempio: testina posizionata inizialmente sulla traccia 20 e coda richieste: [14, 40, 23, 47, 7].

- **FCFS** (First-Come, First-Served): elabora le richieste nell'ordine esatto di arrivo. È equo e privo di starvation, ma altamente inefficiente a causa dei continui spostamenti alternati della testina.
Ordine: 20 → 14 → 40 → 23 → 47 → 7.
- **SSTF** (Shortest Seek Time First): seleziona la richiesta che si trova più vicina alla posizione corrente della testina sul disco. Riduce sensibilmente il tempo medio di seek, ma introduce il grave rischio di starvation per le richieste destinate alle tracce più esterne se continuano ad arrivare richieste concentrate vicino alla posizione corrente della testina.
Ordine: 20 → 23 → 14 → 7 → 40 → 47.
- **SCAN** (Algoritmo dell'Ascensore): la testina si muove continuamente da un'estremità all'altra del disco, soddisfacendo tutte le richieste che incontra lungo il percorso. Una volta raggiunto il fondo (es. traccia 0 o l'ultima traccia), inverte la direzione di marcia. Questo algoritmo elimina il rischio di starvation e rende i tempi di attesa più uniformi.

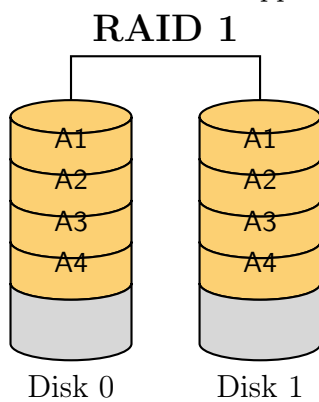
26 RAID: Redundant Array of Independent Disks

La tecnologia RAID combina molteplici dischi rigidi fisici paralleli per presentarli al sistema operativo come un'unica unità logica, con l'obiettivo di incrementare le prestazioni di I/O e/o garantire l'affidabilità tramite la ridondanza dei dati.

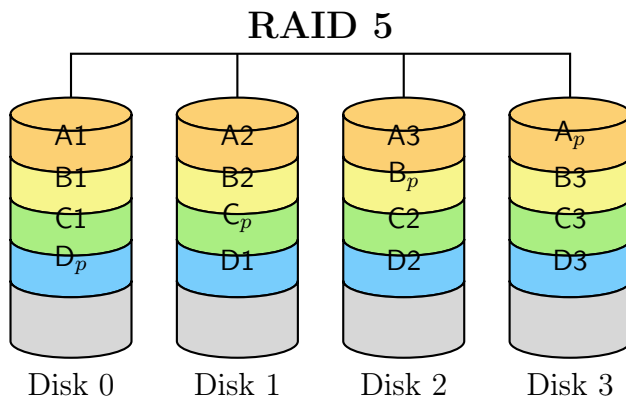
- **RAID 0** (Striping): i dati vengono suddivisi in blocchi (strisce) distribuiti ciclicamente su tutti i dischi dell'array. Vantaggi: elevate prestazioni in lettura e scrittura grazie al parallelismo delle testine. Svantaggi: nessuna ridondanza; se un solo disco si guasta, l'intero set di dati va perduto.



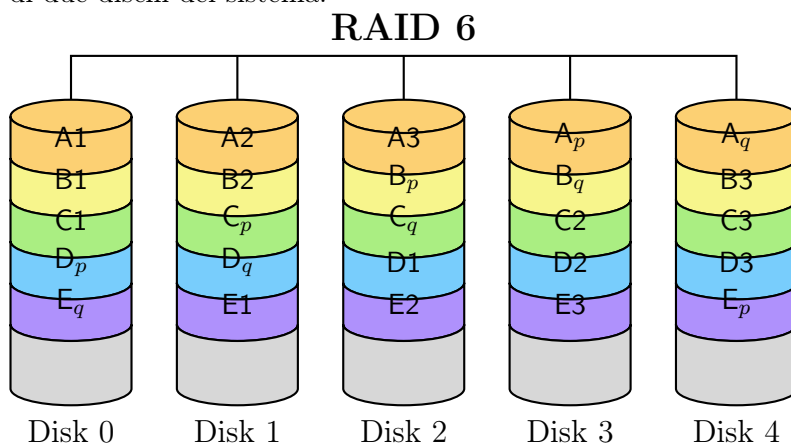
- **RAID 1** (Mirroring): ogni blocco di dati viene duplicato in modo identico e speculare su due o più dischi separati. Vantaggi: massima tolleranza ai guasti (il sistema sopravvive finché resiste almeno un disco del mirror). Svantaggi: efficienza di memorizzazione pari ad appena il 50% e costi raddoppiati.



- **RAID 5** (Striping con parità distribuita): i dati e i blocchi di parità (calcolati tramite operazione logica XOR) vengono distribuiti a rotazione su tutti i dischi dell'array. Vantaggi: consente il ripristino completo dei dati a seguito del guasto di un singolo disco. Lo spazio sprecato per la ridondanza equivale a quello di un solo disco. Svantaggi: ogni operazione di scrittura subisce un overhead dovuto al ricalcolo e aggiornamento della parità.



- **RAID 6** (Striping con doppia parità): evoluzione del RAID 5, calcola due configurazioni indipendenti di parità distribuite su tutti i dischi. Vantaggi: resiste al guasto contemporaneo di due dischi del sistema.



27 Protezione

La protezione è l'insieme dei meccanismi interni al sistema operativo volti a controllare e regolamentare l'accesso alle risorse del sistema (sia fisiche che logiche) da parte di soggetti definiti (processi, utenti). Si distingue dalla **Sicurezza**, che riguarda invece le difese contro minacce ed attacchi esterni.

- **Principio del Minimo Privilegio** (Least Privilege): stabilisce che ogni utente o processo deve operare utilizzando esclusivamente il minor insieme di privilegi e risorse strettamente necessari a completare la sua attività corrente. Riduce l'impatto di bug o azioni malevole.
- **Reference Monitor**: è l'architettura software centralizzata del kernel preposta a intercettare qualsiasi richiesta di accesso a un oggetto da parte di un soggetto. Garantisce che nessuna risorsa possa essere manipolata aggirando le verifiche di autorizzazione.
- **Dominio di Protezione**: definisce l'ambiente in cui opera un processo, specificando le coppie $\langle \text{Oggetto}, \{\text{Insieme Diritti Accesso}\} \rangle$ valide in quel preciso momento.
- **Matrice degli Accessi**: rappresentazione formale in cui le righe sono i domini e le colonne sono gli oggetti. Poiché la matrice è estremamente sparsa, viene implementata in due modi pratici:
 - **ACL** (Access Control List): scomposizione per colonne. Ogni oggetto memorizza la lista dei domini autorizzati con i relativi permessi (es. i permessi dei file in Unix).
 - **Capability List**: scomposizione per righe. Ogni dominio possiede una lista di token o chiavi incontestabili (le capability) che esibisce per accedere agli oggetti. Le capability list risiedono nello spazio protetto del kernel per impedirne la falsificazione da parte dell'utente.